AI Lead Generation Bot for Telegram

What it is, how it works, and why it’s reliable.

What it does

- Finds new leads, scores them, and sends alerts in Telegram
- Checks 15+ sources in parallel
- Uses keyword filtering plus LLM scoring for fit

Design principles

- Freshness: only leads from the last 30 days
- Relevance: two-stage scoring keeps noise low
- Learning: Apply/Skip feedback improves results

Architecture at a glance

- Fetch & parse: HTTP for static sites; headless browser for JS-heavy
- Deduplicate: URL/content hash; drop stale items
- Score: keywords → LLM fit (0–10) + verdict
- Store: SQLite local with optional cloud sync
- Deliver: Telegram with [Apply], [Skip] buttons
- Feedback: Apply/Skip outcomes are recorded and used to refine scoring

Made with GAMMA

CHAPTER 2

MCP Tools (All 5)

All tools are read-only; none change scoring.

- `search_leads(keywords, days, limit)` → list of leads: id, title, source, score, verdict
- `get_lead_details(lead_id)` → full record: url, budget, requirements, risk, reasoning
- `get_scan_summary()` → database counts: total, fresh, score 6+, sent
- `get_profile()` → owner profile text (contact line removed)
- `draft_application(lead_id)` → proposal text draft (generated only; never sent or stored)

Made with GAMMA

CHAPTER 3

Data, Scoring, and Storage

Scoring

- Keyword match: fast filter against profile skills
- LLM fit: 0–10 score + verdict (MATCH / PARTIAL / NO)
- Factors: skills match, budget fit, requirements, risk flags

Storage & Feedback

- Storage: SQLite local; optional cloud sync
- Freshness & dedup: drop items >30 days; hash by URL/content
- Retention: archive after action or age threshold
- Feedback: Apply/Skip outcomes are recorded and used to refine scoring and reporting

Made with GAMMA

CHAPTER 4 — TELEGRAM INTEGRATION & SPECS

Telegram Integration & Technical Specs

Telegram delivery

- Message: title, short summary, score, budget
- Buttons: [Apply] [Skip]
- Actions: recorded as outcomes; used for feedback

Technical specs

- Fetch: HTTP for static; headless browser for JS-heavy
- LLM: configurable; prompted fit scoring with JSON output
- Deploy: single Python service on Linux VPS; cron-scheduled scans
- DB: SQLite for single-user; PostgreSQL for cloud
- Cost control: batch requests and per-source rate limits

Made with GAMMA